

Guía FP Ejercicio

Contador de números positivos

Propósito de la guía. Acompañarte paso a paso para desarrollar el ejercicio de conteo de números positivos, primero en PSeInt y luego en Code::Blocks, cuidando la lógica, la sintaxis y la prueba de escritorio. La idea es que no solo funcione, sino que entiendas por qué funciona.

Resultado esperado

Leer números enteros hasta que el usuario ingrese 0 y contar cuántos de ellos son positivos.

Contexto del ejercicio

En este ejercicio el programa solicita números enteros al usuario de forma repetitiva.

El proceso termina cuando el usuario ingresa 0.

Cada vez que el número ingresado sea mayor que cero, el contador aumenta en uno.

La condición utilizada es:

`num>0`

Y el contador se actualiza mediante:

`cont=cont+1`

1. Seudocódigo para PSeInt

Antes de programar en C, conviene escribir la solución en PSeInt. Aquí se observa la lógica con palabras cercanas al lenguaje natural.

Proceso ContadorPositivos

Definir num, cont Como Entero

cont <- 0

Escribir "Ingresa un numero (0 para terminar): "
Leer num

Mientras num <> 0 Hacer

Si num > 0 Entonces

cont <- cont + 1

```
FinSi
```

```
    Escribir "Ingrese otro numero: "
    Leer num
```

```
FinMientras
```

```
    Escribir "Cantidad de positivos: ", cont
```

```
FinProceso
```

Tip amigable: Imagina que: num guarda cada número ingresado, cont cuenta cuántos números positivos aparecen, el ciclo continúa hasta que el usuario escriba 0.

2. Prueba de escritorio en PSeInt

La prueba de escritorio permite comprobar manualmente qué hace el algoritmo. Para que sea fácil de revisar, se muestra el comportamiento.

num	cont
3	1
-1	1
5	2

Salida esperada:

```
Ingrese un número (0 para terminar): 3
Ingrese otro número: -2
Ingrese otro número: 5
Ingrese otro número: 0
Cantidad de positivos: 2
```

3. Guía para pasar de PSeInt a Code::Blocks con sintaxis validada

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

PSeInt	C / Code::Blocks	Para recordar
Proceso ... FinProceso	int main() { ... return 0; }	Todo programa en C inicia en main.
Definir variables reales	int num, cont;	Los números enteros se declaran con int.
Escribir	Printf();	Printf muestra mensajes y resultados.
Leer	Scanf();	Scanf permite ingresar datos.
Mientras	while	While repite mientras la condición sea verdadera.
num <> 0	num != 0	En C, diferente de se escribe !=0
cont <- cont + 1	cont = cont + 1;	Se utiliza una variable contador.

Si ... Entonces	if (...)	If evalúa condiciones.
-----------------	----------	------------------------

Código validado en C para Code::Blocks

```
#include <stdio.h>

int main()
{
    int num, cont;

    cont = 0;

    printf("Ingrese un numero (0 para terminar): ");
    scanf("%d", &num);

    while(num != 0)
    {
        if(num > 0)
        {
            cont = cont + 1;
        }

        printf("Ingrese otro numero: ");
        scanf("%d", &num);
    }

    printf("Cantidad de positivos: %d", cont);

    return 0;
}
```

Pasos sugeridos en Code::Blocks

1. Crear un nuevo archivo con extensión .c.
2. Copiar el código validado en C.
3. Compilar con Build o F9
4. Ejecutar con Run o con Build and Run.
5. Ingresar números positivos y negativos.
6. Finalizar ingresando 0.
7. Comparar los resultados con la prueba del resultado.

Errores comunes que debes evitar:

- Olvidar inicializar el contador en 0.
- Confundir != con =.
- Olvidar volver a leer num dentro del ciclo.
- Incrementar el contador para números negativos.
- Olvidar el símbolo & en scanf.

4. Rúbrica de evaluación sobre 20 puntos

Esta rúbrica permite valorar el desarrollo completo del ejercicio, desde la comprensión del problema hasta la ejecución en Code::Blocks.

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Comprende correctamente el conteo de números positivos.	Comprende parcialmente la lógica.	Confunde contador y condición de parada.	4 pts
Seudocódigo en PSeInt	Usa correctamente mientras if y contador.	Presenta pequeños errores de sintaxis.	El algoritmo está incompleto.	4 pts
Prueba de escritorio	Comprueba correctamente los resultados.	Presenta una prueba parcial.	No evidencia seguimiento lógico.	4 pts
Traducción a C en Code::Blocks	El código compila y ejecuta correctamente.	Tiene errores menores corregibles.	El código no compila o falla.	5 pts
Presentación y orden	Trabajo limpio y organizado.	Comprensible con pequeños detalles.	Trabajo desordenado.	3 pts

Total: 20 puntos. Para una buena calificación, lo más importante es que exista coherencia entre pseudocódigo, prueba de escritorio, código en C y salida final.

Lista rápida de verificación antes de entregar

- El programa solicita números repetidamente.
- El ciclo termina al ingresar 0.
- Solo se cuentan números positivos.
- El contador inicia en 0.
- El resultado final se muestra correctamente.
- El código compila sin errores.